Tauri + React + Rust Calculator — সম্পূর্ণ গাইড

Tauri + React + Rust Calculator — সম্পূর্ণ গাইড

আর্কিটেকচার

```
React (Frontend/UI)  ←→  Tauri IPC Bridge  ←→  Rust (Backend Logic)
   src/App.jsx              invoke()            src-tauri/main.rs
```

তোমার CLI কোড হুবহু Rust-এ থাকবে — শুধু `stdin` থেকে পড়ার বদলে React UI থেকে ডেটা আসবে।

Step 1: Prerequisites ইন্সটল করো

```
# 1. Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# 2. Node.js (https://nodejs.org) — LTS version ডাউনলোড করো

# 3. Tauri CLI (system dependency আগে লাগবে)
cargo install tauri-cli

# Windows-এ এটাও লাগবে:
# Microsoft C++ Build Tools + WebView2
# https://tauri.app/start/prerequisites/
```

> Tauri-এর official prerequisites: https://tauri.app/start/prerequisites/

Step 2: নতুন Tauri Project তৈরি করো

```
npm create tauri-app@latest

# প্রশ্নের উত্তর:
# Project name: calculator-app
# Frontend language: JavaScript / TypeScript
# Package manager: npm
# UI Template: React
# UI flavor: JavaScript (বা TypeScript)
```

এরপর:

```
cd calculator-app
npm install
```

ফোল্ডার স্ট্রাকচার দেখাবে এরকম:

```
calculator-app/
├── src/                    ← React (Frontend)
│   ├── App.jsx
│   └── main.jsx
├── src-tauri/              ← Rust (Backend)
│   ├── src/
│   │   └── main.rs
│   ├── Cargo.toml
│   └── tauri.conf.json
├── index.html
└── package.json
```

Step 3: Rust Backend — `src-tauri/src/main.rs`

তোমার পুরনো CLI কোড থেকে `input()` ফাংশন বাদ যাবে।

বাকি সব (`add`, `sub`, `mul`, `div`) হুবহু রাখো।

শুধু `#[tauri::command]` দিয়ে একটা নতুন ফাংশন বানাও।

```
// src-tauri/src/main.rs

#![cfg_attr(not(debug_assertions), windows_subsystem = "win
dows")]
```

```rust
fn add(a: i32, b: i32) -> i32 { a + b }
fn sub(a: i32, b: i32) -> i32 { a - b }
fn mul(a: i32, b: i32) -> i32 { a * b }
fn div(a: i32, b: i32) -> i32 {
    if b == 0 { return 0; }  // division by zero check
    a / b
}

// এই ফাংশনটাই React থেকে call হবে
#[tauri::command]
fn calculate(val: i32, a: i32, b: i32) -> String {
    let result = match val {
        1 => add(a, b),
        2 => sub(a, b),
        3 => mul(a, b),
        4 => div(a, b),
        _ => return "Invalid operation".to_string(),
    };
    result.to_string()
}

fn main() {
    tauri::Builder::default()
        .invoke_handler(tauri::generate_handler![calculat
e])  // command register করো
        .run(tauri::generate_context!())
        .expect("error while running tauri application");
}
```

Step 4: React Frontend — `src/App.jsx`

```jsx
// src/App.jsx

import { useState } from "react";
import { invoke } from "@tauri-apps/api/tauri";

function App() {
```

```
  const [a, setA] = useState("");
  const [b, setB] = useState("");
  const [operation, setOperation] = useState(1);
  const [result, setResult] = useState(null);
  const [error, setError] = useState("");

  const operations = [
    { val: 1, label: "Add (+)" },
    { val: 2, label: "Subtract (−)" },
    { val: 3, label: "Multiply (×)" },
    { val: 4, label: "Divide (÷)" },
  ];

  async function handleCalculate() {
    setError("");
    const numA = parseInt(a);
    const numB = parseInt(b);

    if (isNaN(numA) || isNaN(numB)) {
      setError("সঠিক number দিন!");
      return;
    }
    if (operation === 4 && numB === 0) {
      setError("শূন্য দিয়ে ভাগ করা যাবে না!");
      return;
    }

    // Rust-এর calculate() function call হচ্ছে এখানে
    const res = await invoke("calculate", {
      val: operation,
      a: numA,
      b: numB,
    });

    setResult(res);
  }

  return (
```

```
    <div style={styles.container}>
      <h1 style={styles.title}>Calculator</h1>

      <div style={styles.row}>
        <input
          style={styles.input}
          type="number"
          placeholder="প্রথম সংখ্যা (a)"
          value={a}
          onChange={(e) => setA(e.target.value)}
        />
        <input
          style={styles.input}
          type="number"
          placeholder="দ্বিতীয় সংখ্যা (b)"
          value={b}
          onChange={(e) => setB(e.target.value)}
        />
      </div>

      <div style={styles.opRow}>
        {operations.map((op) => (
          <button
            key={op.val}
            style={{
              ...styles.opBtn,
              background: operation === op.val ? "#185FA5"
: "transparent",
              color: operation === op.val ? "#fff" : "inher
it",
            }}
            onClick={() => setOperation(op.val)}
          >
            {op.label}
          </button>
        ))}
      </div>
```

```
      <button style={styles.calcBtn} onClick={handleCalcula
te}>
        Calculate
      </button>

      {error && <p style={styles.error}>{error}</p>}

      {result !== null && !error && (
        <div style={styles.result}>
          Result = <strong>{result}</strong>
        </div>
      )}
    </div>
  );
}

const styles = {
  container: {
    maxWidth: 420,
    margin: "60px auto",
    padding: "32px",
    borderRadius: 16,
    border: "1px solid #e0e0e0",
    fontFamily: "sans-serif",
  },
  title: { textAlign: "center", marginBottom: 24, fontSize:
24 },
  row: { display: "flex", gap: 12, marginBottom: 16 },
  input: {
    flex: 1,
    padding: "10px 14px",
    borderRadius: 8,
    border: "1px solid #ccc",
    fontSize: 16,
  },
  opRow: { display: "flex", gap: 8, marginBottom: 20, flexW
rap: "wrap" },
  opBtn: {
```

```
    flex: 1,
    padding: "8px 4px",
    borderRadius: 8,
    border: "1px solid #ccc",
    cursor: "pointer",
    fontSize: 13,
    minWidth: 80,
  },
  calcBtn: {
    width: "100%",
    padding: "12px",
    background: "#185FA5",
    color: "#fff",
    border: "none",
    borderRadius: 8,
    fontSize: 16,
    cursor: "pointer",
  },
  error: { color: "#E24B4A", textAlign: "center", marginTo
p: 12 },
  result: {
    textAlign: "center",
    marginTop: 20,
    fontSize: 22,
    padding: 16,
    background: "#EAF3DE",
    borderRadius: 8,
  },
};

export default App;
```

Step 5: Run করো

```
# Development mode (hot reload সহ):
npm run tauri dev
```

```
# Production build (installer তৈরি):
npm run tauri build
```

`tauri build` করলে `src-tauri/target/release/bundle/` ফোল্ডারে installer পাবে।

কী কী পরিবর্তন হলো তোমার কোড থেকে

CLI কোড	Tauri কোড
`input()` → stdin পড়তো	React `<input>` → state থেকে পড়ে
`match val { 1=>add... }`	হুবহু একই — `calculate()` ফাংশনে
`println!("Result = {}")`	`result.to_string()` → React-এ দেখায়
Terminal চালাতে হতো	Desktop window খুলবে

সমস্যা হলে

```
# Rust dependency update
cd src-tauri && cargo update

# Node modules পরিষ্কার করো
rm -rf node_modules && npm install
```

Platform-specific সমস্যার জন্য: https://tauri.app/start/prerequisites/